

Health Care

심근경색/협심증 발병 위험도 예측 및 건강 정보 웹 서비스 개발

2차 프로젝트 1조

강현기 곽민재 김민성 박성준

발표자 박성준

목차

01 프로젝트 개요

주제 선정 배경
프로젝트 목표
프로젝트 일정
팀원소개

02 데이터 분석

개발 환경
데이터 수집 및 전처리
분석
분석 시각화
모델링
분석 결론

03 웹 서비스 개발

개발환경
시스템 아키텍처
API 인터페이스 명세
주요 기능
시연

04 프로젝트 결론

결론 및 제언
활용 방안
기대효과
Q&A

1. 프로젝트 개요

주제 선정 배경

프로젝트 목표

팀원 소개

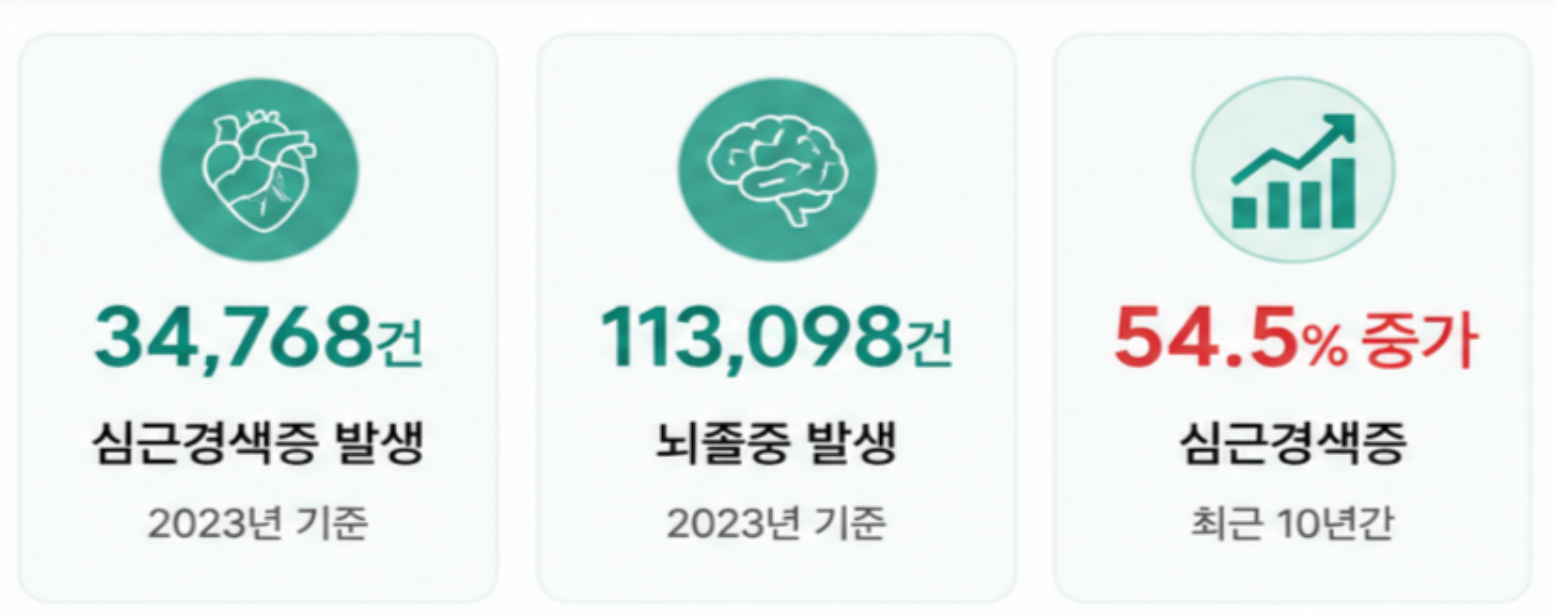
프로젝트 일정

Keyword

프로젝트 개요

주제 선정 배경

높은 발생 규모와 증가 추세



“**"심장질환은 최근 10년간
암에 이어 국내 사망원인 2위"**”

**발병 시 응급치료·입원·장기치료가 필요해
개인과 가계의 의료비 부담이 증가**



심근경색증 34,768건, 뇌졸중 113,098건 발생 (질병관리청, 『2023 심뇌혈관질환 발생통계』)
최근 10년간 심근경색증 54.5%, 뇌졸중 9.5% 증가(질병관리청, 『2024년 10년 새 심뇌혈관질환 발생 증가 보도자료』)

프로젝트 개요

주제 선정 배경

의료기관과 소비자 간 정보 비대칭 해소



기존 방식

- 병원 검사 의존
- 전문 해석 필요



소비자 한계

- 위험 신호 인지 어려움
- 자가 판단 어려움

조기 증상 인지율

심근경색	51.5%
뇌졸중	60.7%



개선 방향

- 위험도 예측 제공
- 병원 방문 전 점검 가능

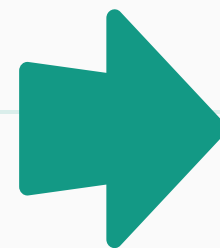
정부도 '건강정보 고속도로'로 정보 접근성 개선 추진
→ 정책 기반 비대칭 완화

프로젝트 개요

프로젝트 목표

위험도 예측 기능 구현

- 간단한 검진 데이터 입력
- 심근경색 및 협심증 발병 확률 예측
- 예측 결과를 수치와 시각자료로 제공



건강 정보 제공

- 위험 요인별 코멘트 제공
- 유저 간 정보공유
- 병원 방문 전 자가 점검 지원

프로젝트 개요

프로젝트 일정

		Day 1 (05.14)	Day 2 (05.15)	Day 3 (05.18)	Day 4 (05.19)	Day 5 (05.20)	Day 6 (05.21)	Day 7 (05.22)	Day 8 (05.26)	
	STEP 1	주제 선정 및 계획								
	STEP 2		머신러닝 모델링(데이터 수집/전처리/분석/모델링)							
	STEP 3		데이터 베이스 설계							
	STEP 4		Backend(Java - Springboot, Python-Flask), Frontend(React) 개발							
	STEP 5							프로젝트 관련 자료 제작		

프로젝트 개요

조원 소개



조장

강현기

Backend(SpringBoot)
Frontend(React)
프로젝트 자료 제작

조원

곽민재

DB(Oracle)
Backend(SpringBoot)
Frontend(React)

조원

김민성

Frontend(React)
프로젝트 자료 제작

조원

박성준

Backend(Flask, SpringBoot)
Frontend(React)
머신러닝

2. 데이터 분석

개발환경

데이터 수집 및
전처리

분석

모델링

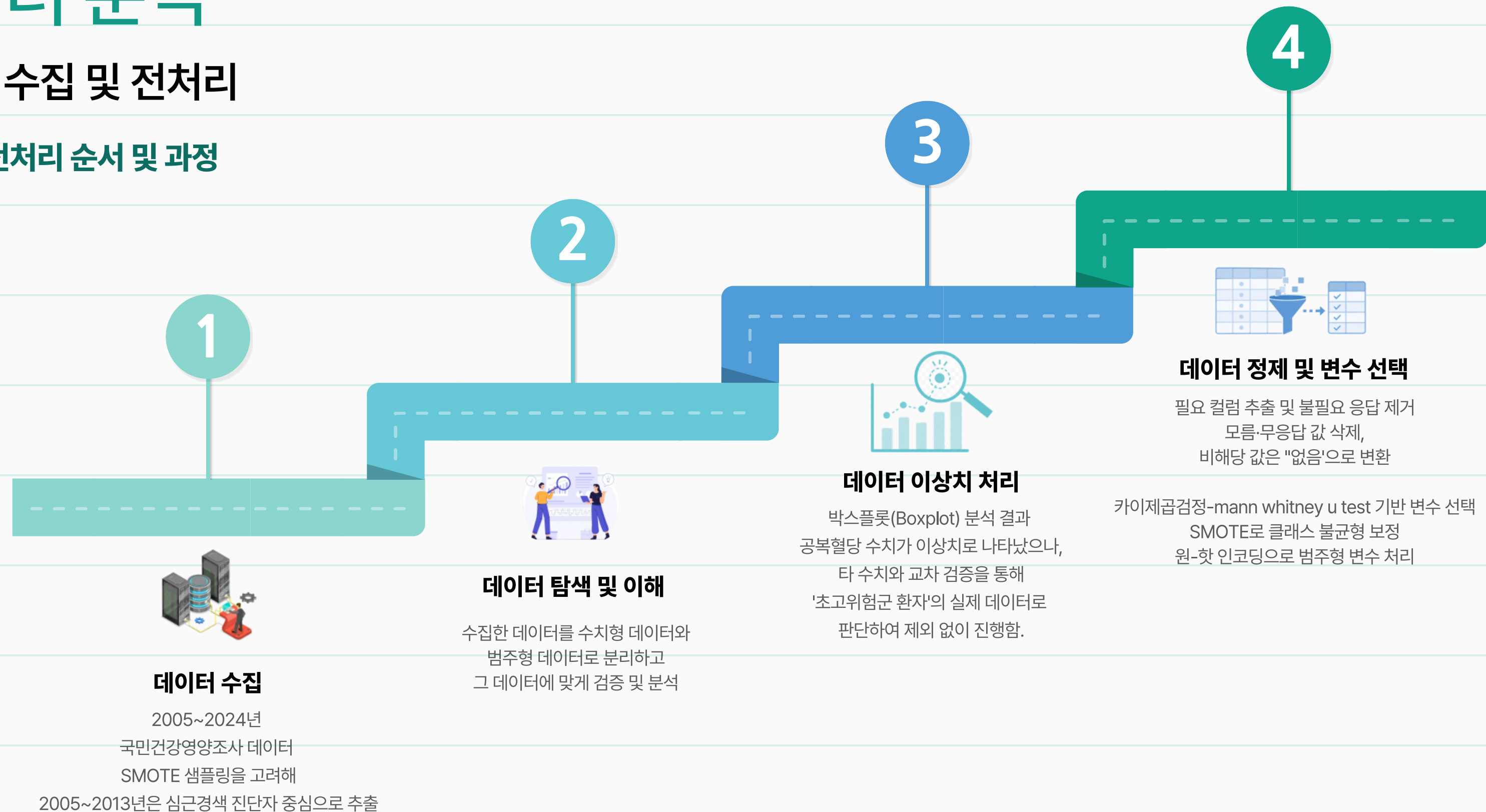
분석 결론

한계점

데이터 분석

데이터 수집 및 전처리

데이터 전처리 순서 및 과정



데이터 분석

데이터 수집 및 전처리

데이터 수집

1. 데이터 출처

질병관리청 국민건강영양조사

- 질병 관리청에서 제공하는 공공 보건 의료 데이터
- 건강검진, 질병 이력, 생활습관 관련 항목 포함
- 신뢰성 있는 공공 데이터 활용



2. 수집 데이터 형태



sas7bdat 파일

모델 학습 및 분석에
사용한 원본 데이터



데이터 정의서

변수명, 항목 의미,
응답코드 및 결측값
의미 확인에 사용함



3. 주요 수집 항목



인구통계 정보

성별, 연령 등



건강검진 정보

혈압, 혈당, 콜레스테롤 등



질병 관련 정보

심근경색, 협심증 진단 여부 등



생활습관 정보

흡연, 음주, 운동 여부 등

데이터 분석

데이터 수집 및 전처리

데이터 선정 이유 및 근거

논문 제목	논문 URL
성인 관상동맥심장질환 고위험군 환자의 급성심근경색 발생 및 사망에 영향을 미치는 요인	https://www.dbpia.co.kr/pdf/cpViewer?nodeId=T14188393
한국 75세 이상 노인에서 심근경색 발생에 체질량지수와 허리둘레가 미치는 영향	https://www.dbpia.co.kr/pdf/cpViewer?nodeId=T15944683
사회경제적 불평등과 심뇌혈관 질환	https://jogh.org/2024/jogh-14-04210

데이터 분석

데이터 수집 및 전처리

선정된 데이터들



범주형 데이터

12개

변수명	실제 항목 이름
DI1_dg	고혈압 진단여부
DI2_dg	이상지질혈증 의사진단
DE1_dg	당뇨병 의사진단 여부
DI3_dg	뇌졸중 의사 진단 여부
HE_HP	고혈압 유병여부
BS1_1	평생 일반담배 흡연 여부
BD1_11	1년간 음주 빈도
BD2_1	한번에 마시는 음주량
PA_aerobic	유산소 신체활동 실천율
sex	나이
edu	교육수준
ho_incm	소득 사분위수



수치형 데이터

6개

변수명	실제 항목 이름
HE_glu	공복혈당
HE_HbA1c	당화혈색소
HE_chol	총콜레스테롤
HE_BMI	체질량지수
BE8_1	앉아서 보내는 시간
age	나이

데이터 분석

데이터 수집 및 전처리

데이터 전처리

데이터 통합 및 심근경색 및 협심증에 걸린사람에 대한 추가 데이터 추가

```
# 1. 모든 데이터프레임을 리스트로 묶습니다.
df_list = [df14, df15, df16, df17, df18, df19, df20, df21, df22, df23, df24]

# 2. 한 번에 위아래로 합칩니다.
df_total = pd.concat(df_list, ignore_index=True)

# 1. 모든 데이터프레임을 리스트로 묶습니다.
df_list1 = [df13,df12,df11,df10,df9 ,df8 ,df7 ,df5 ]

# 2. 한 번에 위아래로 합칩니다.
df_total1 = pd.concat(df_list, ignore_index=True)
```

```
### "DI4_dg","DI5_dg" 이 두개 컬럼을 종속변수로 둠
# DI4_dg = 심근경색증 또는 협심증 의사 진단 여부(0.없음, 1.있음, 8.비해당)
# DI5_dg = 심근경색증 의사 진단 여부(0.없음, 1.있음, 8.비해당, 9.모름, 무응답)

data = df_total[["DI4_dg", "DI5_dg", "DI1_dg", "DI2_dg", "DE1_dg", "DI3_dg", "HE_HP", "HE_glu", "HE_HbA1c", "HE_chol", "HE_BMI", "HE_wc",
                 "BS1_1", "BD1_11", "BD2_1", "BE8_1", "pa_aerobic",
                 "sex", "age", "edu", "ho_incm"]]
```

```
# 심근경색걸린사람 추가 데이터
df_one_data = df_total1[df_total1["DI4_dg"] == 1].copy()

df_total2 = df_one_data[["DI4_dg", "DI5_dg", "DI1_dg", "DI2_dg", "DE1_dg", "DI3_dg", "HE_HP", "HE_glu", "HE_HbA1c", "HE_chol", "HE_BMI", "HE_wc",
                        "BS1_1", "BD1_11", "BD2_1", "BE8_1", "pa_aerobic",
                        "sex", "age", "edu", "ho_incm"]].dropna()
```

데이터 분석

데이터 수집 및 전처리

데이터 전처리

범주형 데이터의 8(비해당),
9(무응답)에 대한 데이터를
8은 없음(0 또는 1), 9는 삭제 처리
결측치는 삭제 처리 이후 SMOTE기법 활용

```
# -----> 8을 0으로 바꿈 9의 데이터는 삭제
col_replace = ["DI1_dg", "DI2_dg", "DE1_dg", "DI3_dg"]
# -----> 8을 1으로 바꿈 9의 데이터는 삭제
col_replace2 = ["BD1_11"]
# -----> 9의 데이터는 삭제
col_replace3 = ["BD2_1"]

for col in col_replace:
    if col in data.columns:
        # 비해당 → 0
        data[col] = data[col].replace(8, 0)
        # 모름/무응답 → 행 삭제
        data = data[data[col] != 9]

for col in col_replace2:
    if col in data.columns:
        data[col] = data[col].replace(8, 1)
        data = data[data[col] != 9]
data = data[data["BD2_1"] != 9]

# -----> 8을 0으로 바꿈 9의 데이터는 삭제
col_replace = ["DI1_dg", "DI2_dg", "DE1_dg", "DI3_dg"]
# -----> 8을 1으로 바꿈 9의 데이터는 삭제
col_replace2 = ["BD1_11"]
# -----> 9의 데이터는 삭제
col_replace3 = ["BD2_1"]

for col in col_replace:
    if col in df_total2.columns:
        # 비해당 → 0
        df_total2[col] = df_total2[col].replace(8, 0)
        # 모름/무응답 → 행 삭제
        df_total2 = df_total2[df_total2[col] != 9]

for col in col_replace2:
    if col in df_total2.columns:
        df_total2[col] = df_total2[col].replace(8, 1)
        df_total2 = df_total2[df_total2[col] != 9]
df_total2 = df_total2[df_total2["BD2_1"] != 9]
```

```
data = data.dropna()
```

```
# 1. 종속변수 독립변수 데이터를 분리
df_ind = datall[selected_features].copy() # 앞에서 통계적으로 살아남은 변수들만 사용!
df_dep = datall["DI4_dg"].copy()

# 2. train/test 데이터 분리 (stratify 추가로 비율 보존)
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    df_ind, df_dep, test_size=0.2, random_state=42, stratify=df_dep
)

# 3. 다시 합치기 (Train 내부 샘플링용)
train_df = pd.concat([X_train_raw, y_train], axis=1)

# 4. train 데이터만 내부에서 언더샘플링 진행
df_0 = train_df[train_df["DI4_dg"] == 0]
df_1 = train_df[train_df["DI4_dg"] == 1]

n_samples = min(10000, len(df_0))
df_0_downsampled = df_0.sample(n=n_samples, random_state=42)

# 5. 합치기
df_combined = pd.concat([df_0_downsampled, df_1], axis=0)

# 6. smote 하기 전 다시 종속, 독립 분리
X_train_semi = df_combined.drop("DI4_dg", axis=1)
y_train_semi = df_combined["DI4_dg"]

# SMOTE 및 모델 학습을 위한 결측치 처리 및 원핫 인코딩
for col in X_train_semi.columns:
    if col in categorical_features:
        # 범주형은 최빈값으로 채우기
        most_freq = X_train_semi[col].mode()[0]
        X_train_semi[col] = X_train_semi[col].fillna(most_freq)
        X_test_raw[col] = X_test_raw[col].fillna(most_freq)
    elif col in numerical_features:
        # 수치형은 중앙값으로 채우기
        med_val = X_train_semi[col].median()
        X_train_semi[col] = X_train_semi[col].fillna(med_val)
        X_test_raw[col] = X_test_raw[col].fillna(med_val)

# 2) 원핫 인코딩 (문자/카테고리 데이터를 0과 1의 숫자로 변환)
actual_cate_cols = [c for c in categorical_features if c in X_train_semi.columns]
X_train_encoded = pd.get_dummies(X_train_semi, columns=actual_cate_cols, drop_first=True)
X_test_encoded = pd.get_dummies(X_test_raw, columns=actual_cate_cols, drop_first=True)

# Train과 Test의 컬럼 순서 및 개수를 완벽히 일치시킵니다.
X_train_encoded, X_test_encoded = X_train_encoded.align(X_test_encoded, join='left', axis=1, fill_value=0)

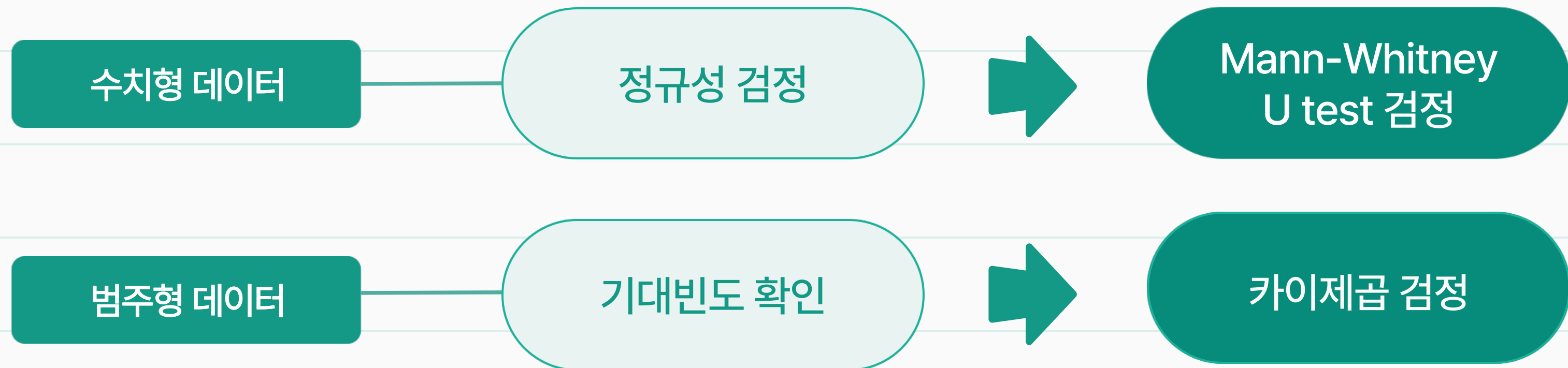
# 7. 결측치와 인코딩이 완료된 데이터로 SMOTE 진행
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_encoded, y_train_semi)

# 결과 확인
print("--- 최종 가공된 Train 데이터의 클래스 비율 ---")
print(y_train_resampled.value_counts())
```

데이터 분석

데이터 수집 및 전처리

데이터별 검정



데이터 분석

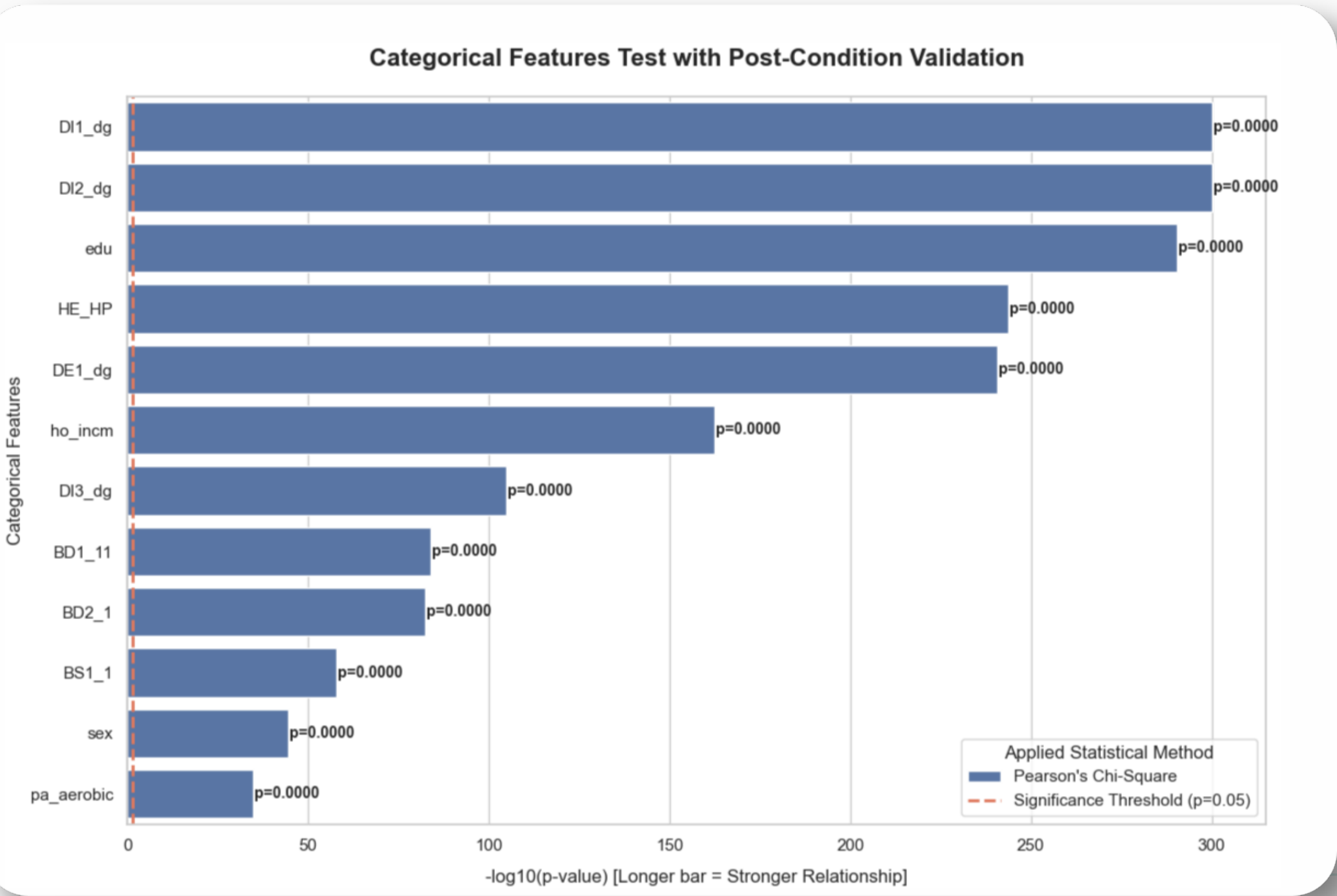
데이터 수집 및 전처리

데이터별 검정(범주형데이터)

범주형 데이터

기대빈도 확인

오른쪽 그래프와 같이
범주형 데이터의 모든 컬럼들의
기대빈도의 수치가 5이상
넘는 것을 확인할 수 있음
따라서, 카이제곱 검정이 가능함



데이터 분석

분석

데이터별 검정(수치형데이터)

수치형 데이터

정규성 검정

수치형 변수의 그룹 간 차이 검정을 위해
정규성 여부를 확인

정규성 검정결과 정규분포를 따르지 않는것으로
확인 되어 Mann-Whitney U test로 진행

--- 수치형 변수 검정 결과 ---

[HE_glu] 정규성 검정:

[DI4_dg=0] Kolmogorov-Smirnov p=0.0000 → 정규분포 X

[DI4_dg=1] Shapiro-Wilk p=0.0000 → 정규분포 X

→ 사용 검정: Mann-Whitney U test, p-value: 0.0000 유의미

[HE_HbA1c] 정규성 검정:

[DI4_dg=0] Kolmogorov-Smirnov p=0.0000 → 정규분포 X

[DI4_dg=1] Shapiro-Wilk p=0.0000 → 정규분포 X

→ 사용 검정: Mann-Whitney U test, p-value: 0.0000 유의미

[HE_chol] 정규성 검정:

[DI4_dg=0] Kolmogorov-Smirnov p=0.0000 → 정규분포 X

[DI4_dg=1] Shapiro-Wilk p=0.0000 → 정규분포 X

→ 사용 검정: Mann-Whitney U test, p-value: 0.0000 유의미

[HE_BMI] 정규성 검정:

[DI4_dg=0] Kolmogorov-Smirnov p=0.0000 → 정규분포 X

[DI4_dg=1] Shapiro-Wilk p=0.0000 → 정규분포 X

→ 사용 검정: Mann-Whitney U test, p-value: 0.0000 유의미

[HE_wc] 정규성 검정:

[DI4_dg=0] Kolmogorov-Smirnov p=0.0000 → 정규분포 X

[DI4_dg=1] Shapiro-Wilk p=0.0000 → 정규분포 X

→ 사용 검정: Mann-Whitney U test, p-value: 0.0000 유의미

[BE8_1] 정규성 검정:

[DI4_dg=0] Kolmogorov-Smirnov p=0.0000 → 정규분포 X

[DI4_dg=1] Shapiro-Wilk p=0.0000 → 정규분포 X

→ 사용 검정: Mann-Whitney U test, p-value: 0.0000 유의미

[age] 정규성 검정:

[DI4_dg=0] Kolmogorov-Smirnov p=0.0000 → 정규분포 X

[DI4_dg=1] Shapiro-Wilk p=0.0000 → 정규분포 X

→ 사용 검정: Mann-Whitney U test, p-value: 0.0000 유의미

데이터 분석

분석

변수 검정 과정 결과

--- 범주형 변수 검정 결과 ---

```
DI1_dg p-value: 0.0000
DI2_dg p-value: 0.0000
DE1_dg p-value: 0.0000
DI3_dg p-value: 0.0000
HE_HP p-value: 0.0000
BS1_1 p-value: 0.0000
BD1_11 p-value: 0.0000
BD2_1 p-value: 0.0000
pa_aerobic p-value: 0.0000
sex p-value: 0.0000
edu p-value: 0.0000
ho_incm p-value: 0.0000
```

--- 수치형 변수 검정 결과 ---

```
HE_glu p-value: 0.0000
HE_HbA1c p-value: 0.0000
HE_chol p-value: 0.0000
HE_TG p-value: 0.2664
HE_BMI p-value: 0.0000
HE_wc p-value: 0.0000
BE8_1 p-value: 0.0000
age p-value: 0.0000
```

최종 선택된 유의미한 독립변수들: ['DI1_dg', 'DI2_dg', 'DE1_dg', 'DI3_dg', 'HE_HP', 'BS1_1', 'BD1_11', 'BD2_1', 'pa_aerobic', 'sex', 'edu', 'ho_incm', 'HE_glu', 'HE_HbA1c', 'HE_chol', 'HE_BMI', 'HE_wc', 'BE8_1', 'age']

범주형 변수는 카이제곱 검정, 수치형 변수는 Mann-Whitney U test를 통해 검증
그 결과 모든 변수들이 유의미하다는 것을 확인

데이터 분석

모델링

머신러닝 모델 선별 과정

모델 학습 결과

모델을 Random Forest, XGBoost, LightGBM을 선택한 이유

LightGBM : 불균형 데이터에서 살려낸 소수클래스(심근경색 발병)의 미세한 특성을 잘 잡아냄

XGBoost : 의학 데이터처럼 변수간의 복잡한 상호작용이 있을때 트리를 정교하게 짜서 잡아내는 능력이 좋음

Random Forest : 위의 모델들이 샘플링된 데이터에 과적합이 나고 있는지에 대한 기준으로 잡음

비교 결과

XGBoost는 과적합 가능성이 낮고 전반적으로 안정적인 성능을 보임

LightGBM는 학습 성능이 다소 미세하거나 불안정향 경향을 보임

Random Forest 학습 시작

	precision	recall	f1-score	support
정상(0)	0.99	0.85	0.91	9100
심근경색(1)	0.25	0.79	0.38	555
accuracy			0.85	9655
macro avg	0.62	0.82	0.65	9655
weighted avg	0.94	0.85	0.88	9655

ROC-AUC Score: 0.9046

XGBoost 학습 시작

	precision	recall	f1-score	support
정상(0)	0.98	0.93	0.95	9100
심근경색(1)	0.39	0.75	0.51	555
accuracy			0.92	9655
macro avg	0.68	0.84	0.73	9655
weighted avg	0.95	0.92	0.93	9655

ROC-AUC Score: 0.9326

LightGBM 학습 시작

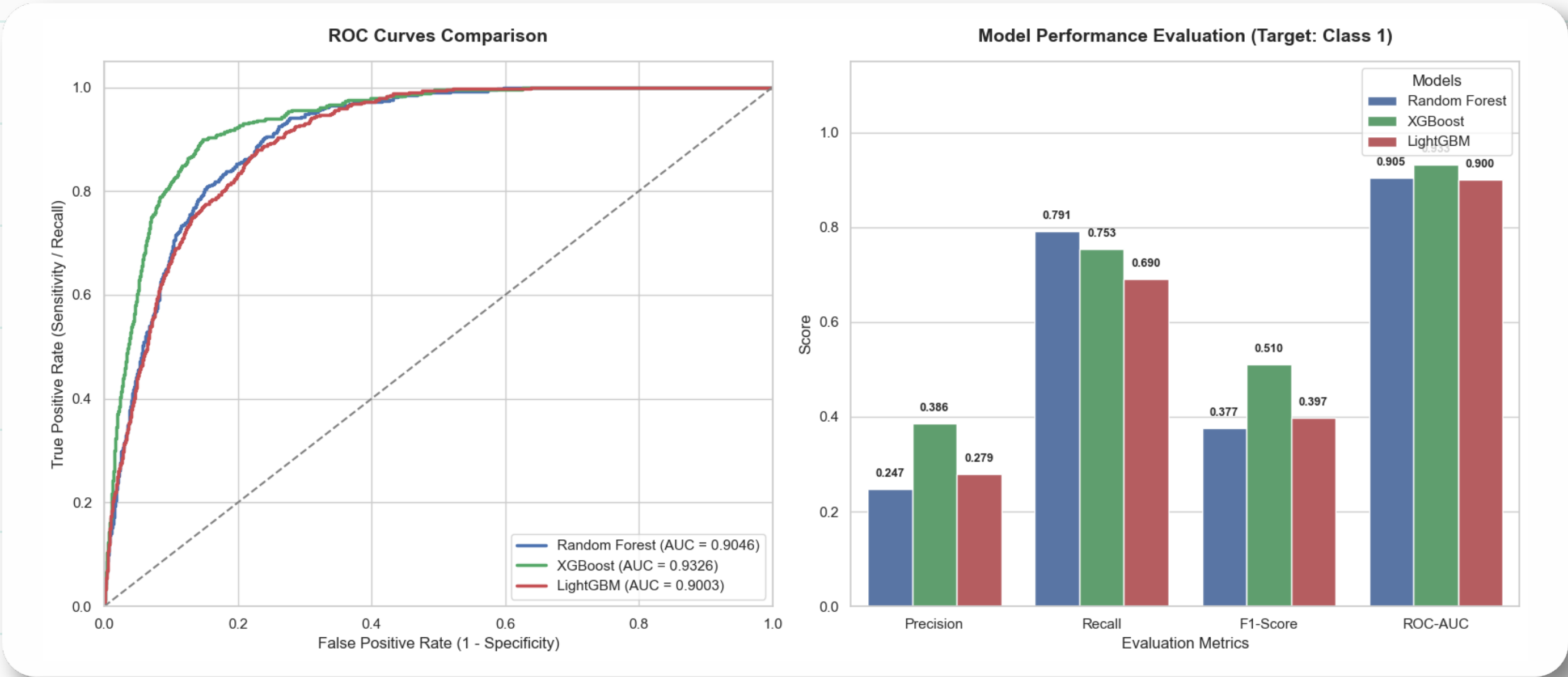
	precision	recall	f1-score	support
정상(0)	0.98	0.89	0.93	9100
심근경색(1)	0.28	0.69	0.40	555
accuracy			0.88	9655
macro avg	0.63	0.79	0.67	9655
weighted avg	0.94	0.88	0.90	9655

ROC-AUC Score: 0.9003

데이터 분석

모델링

모델 선정 과정

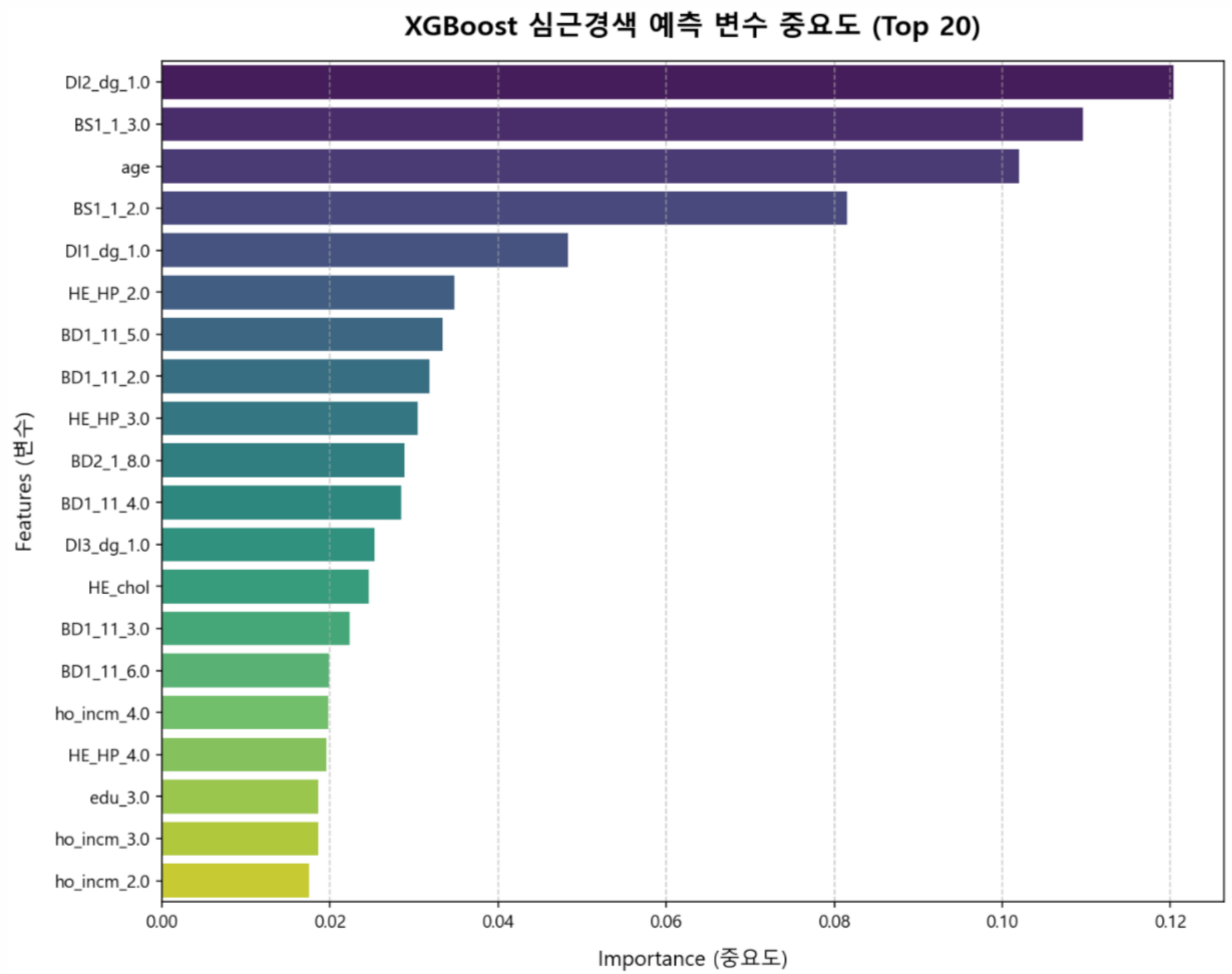


데이터 분석

분석

XGB 변수 중요도

학습된 모델의 변수 중요도를 확인한 결과
이상지질혈증 의사진단 여부(DI2_dg),
평생 일반담배 흡연 여부(BS1_1), 연령(age)
컬럼들이 예측에 주요한 변수로 작용함을 확인함



데이터 분석

모델링

머신러닝 모델링

GridSearchCV를 통해 XGBoost의
최적 하이퍼파라미터를 탐색한 결과,

최고 ROC-AUC 0.9801,
최종 정확도 0.90의 성능을 확인함

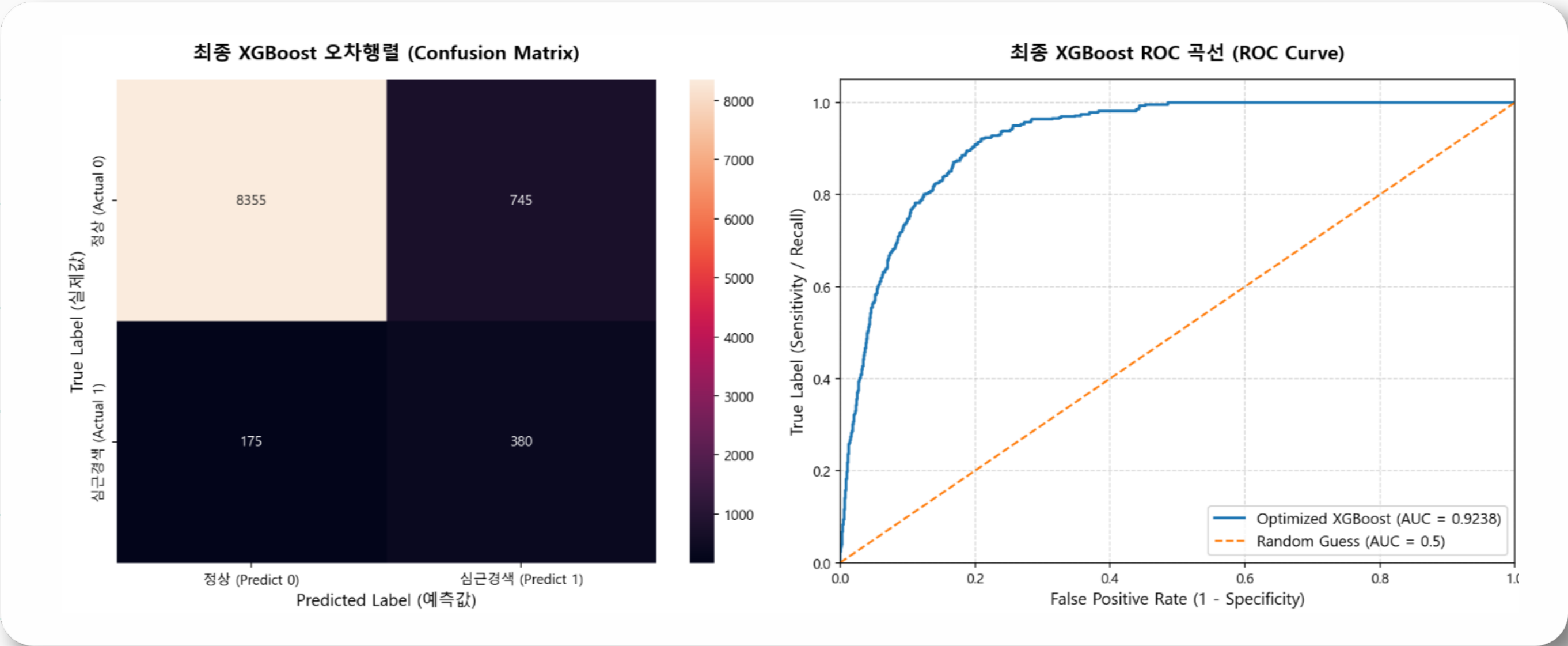
최고 ROC-AUC 점수: 0.9801
최적의 하이퍼파라미터 조합:
{'colsample_bytree': 1.0, 'learning_rate': 0.1, 'max_depth': 6, 'min_child_weight': 1, 'n_estimators': 200, 'subsample': 0.8}

===== 🏆 최적화 XGBoost 최종 성능 평가 =====				
	precision	recall	f1-score	support
정상(0)	0.98	0.92	0.95	9100
심근경색(1)	0.34	0.68	0.45	555
accuracy			0.90	9655
macro avg	0.66	0.80	0.70	9655
weighted avg	0.94	0.90	0.92	9655

데이터 분석

모델링

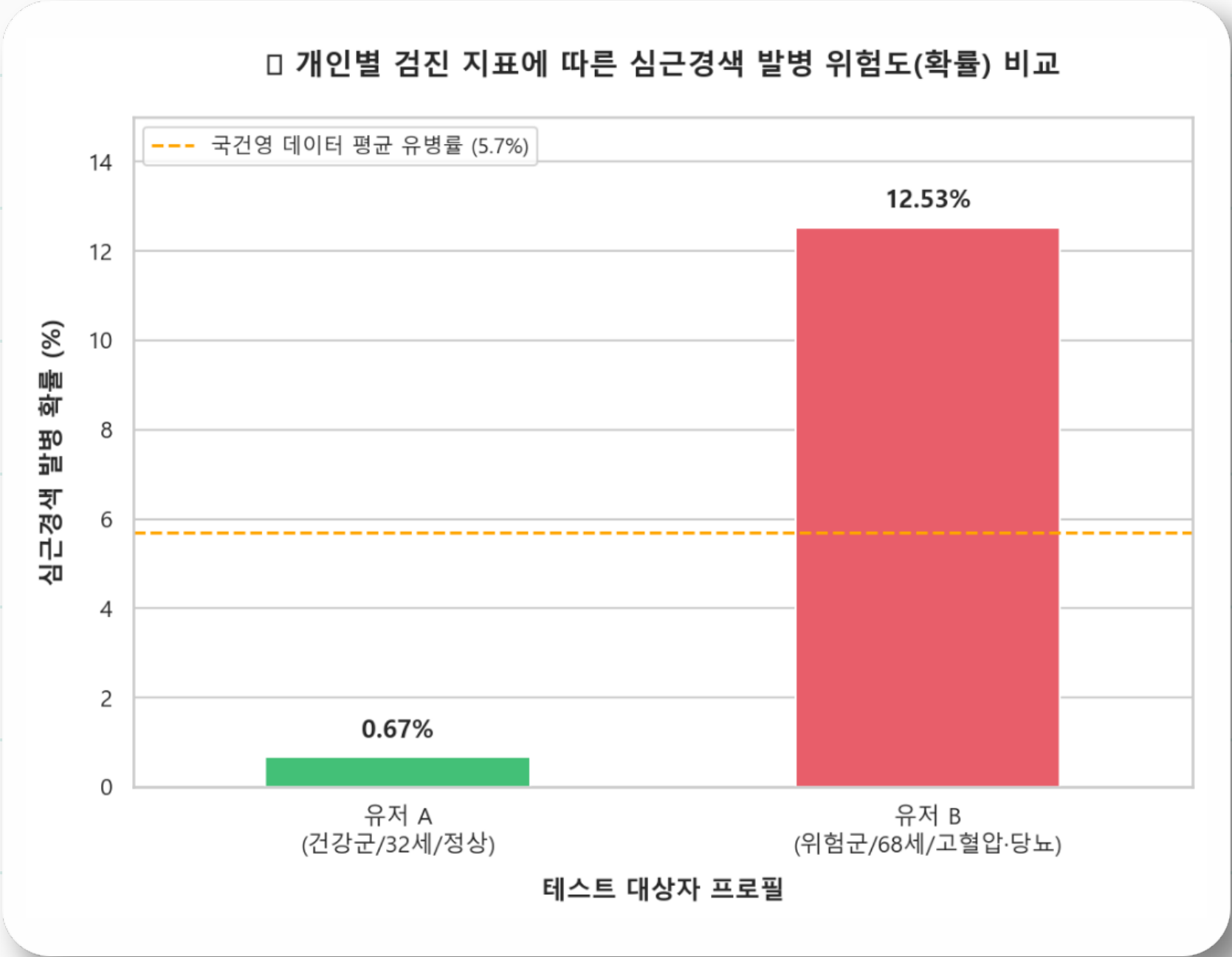
최종오차행렬 및 ROC곡선



데이터 분석

모델링

모델 평가 결과



모델 테스트 결과

발병 확률

실제 평균 유병률

"건강한 사람의 임의 데이터 "
"위험한 사람의 임의 데이터"



"건강한 사람 => 0.67% "
"위험한 사람 => 12.53% "



국건영 데이터 평균 유병률 (5.7%)

데이터 분석

분석 결론

최적화 XGBoost 성능 요약

테스트 데이터 9,655명(정상 9,100명 / 심근경색 555명)을 기준으로 평가한 결과
정상인 사람에 대해서는
0.98로 높은 정확도와 0.92의 높은 재현율을 보여줌
심근경색 및 협심증이 있는 사람에 대해서는
0.34의 정확도와 0.68의 재현률로 많이 부족한 모습을 보여줌

실제 심근경색 환자 중 약 68%를 탐지
→ 위험군 선별에는 활용 가능하지만,
오탐을 줄이기 위한 추가 개선이 필요함.

데이터 구성

전체 데이터 : 9,655명
정상(0) : 9,100명
심근경색(1) : 555명 (5.7%)



분류 성능

	precision	recall	f1-score
정상(0) :	0.98	0.92	0.95
심근경색(1) :	0.34	0.68	0.45

3. 웹 서비스 개발

개발환경

시스템
아키텍처

API 인터페이스
명세

주요 기능

시연

분석 경로
Keyword

웹 서비스 개발

개발 환경



Frontend

사용자 인터페이스 구현



CSS HTML5 JavaScript



Backend

서버 및 API 개발





Database

데이터 저장 및 관리

Oracle DBMS



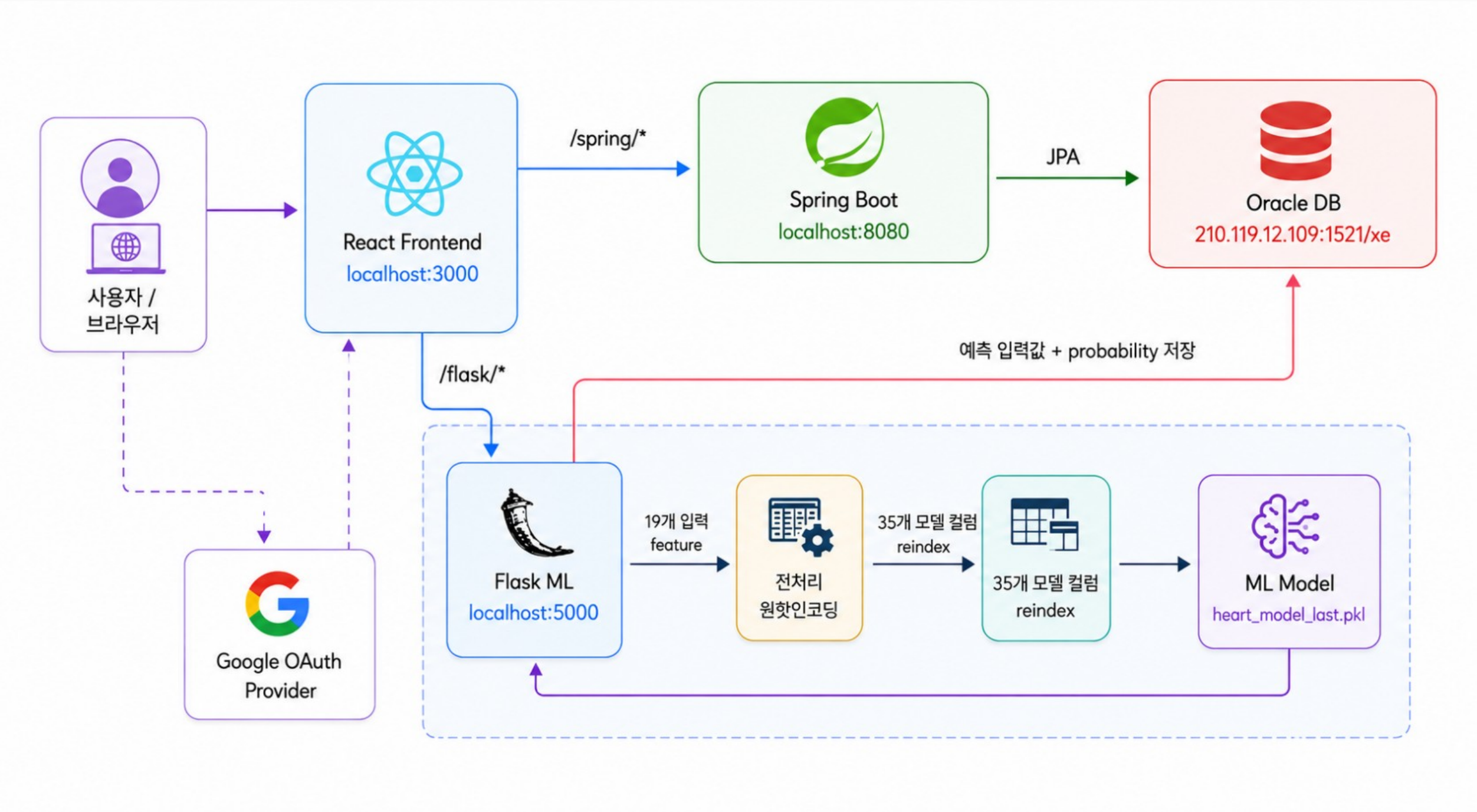
Backend(머신러닝)

머신러닝 모델 저장 및 관리



웹 서비스 개발

시스템 아키텍처



웹 서비스 개발

API 인터페이스 명세 (공통)

서버	Base URL	프론트 호출 Prefix
Spring Boot	http://localhost:8080	/spring
Flask	http://localhost:5000	/flask

웹 서비스 개발

API 인터페이스 명세(SpringBoot - MemberSpringBootAPI)

기능	Method endpoint	요청 형식	응답 형식
회원 연결 확인	GET /member/check	없음	String
회원 전체 조회	GET /member/list	없음	Member[]
회원 상세 조회	GET /member/view/{mem_id}	Path: mem_id	Member
회원 등록	POST /member/insert	Body: Member JSON	성공: Member, 중복: 409 String
회원 수정	PUT /member/update	Body: Member JSON	Member 또는 null
회원 삭제	DELETE /member/delete/{mem_id}	Path: mem_id	204 No Content

웹 서비스 개발

API 인터페이스 명세(SpringBoot - CommunitySpringBootAPI)

기능	Method endpoint	요청 형식	응답 형식
게시판 연결 확인	GET /community/check	없음	String
게시글 목록 조회	GET /community/list?mem_id=	Query 선택: mem_id	Community[]
게시글 상세 조회	GET /community/view/{com_id}	Path: com_id	Community
게시글 등록	POST /community/insert	Body: Community JSON	성공: Community, 실패: 409 String
게시글 수정	PUT /community/update	Body: Community JSON	Community 또는 null
게시글 삭제	DELETE /community/delete/{com_id}	Path: com_id	204 No Content
좋아요 증가	POST /community/like/{com_id}	Path: com_id	Community
인기글 조회	GET /community/top_list	없음	Community[]
내 게시글 조회	GET /community/board/my/{mem_id}	Path: mem_id	Community[]

웹 서비스 개발

API 인터페이스 명세(SpringBoot - CommentSpringBootAPI)

기능	Method endpoint	요청 형식	응답 형식
댓글 목록 조회	GET /comment/list/{comid}	Path: comid	Comment[]
댓글 등록	POST /comment/insert	Body: Comment JSON	Comment
댓글 수정	PUT /comment/update	Body: Comment JSON	Comment 또는 null
댓글 삭제	DELETE /comment/delete? comid=&memid=&commentcreated=	Query: comid, memid, commentcreated	String

웹 서비스 개발

API 인터페이스 명세(SpringBoot - InquirySpringBootAPI)

기능	Method endpoint	요청 형식	응답 형식
문의 연결 확인	GET /inquiry/check	없음	String
문의 전체 조회	GET /inquiry/list	없음	Inquiry[]
문의 상세 조회	GET /inquiry/view/{inq_id}	Path: inq_id	Inquiry
문의 등록	POST /inquiry/insert	Body: Inquiry JSON	성공: Inquiry, 실패: 409 String

웹 서비스 개발

API 인터페이스 명세(SpringBoot - ModelDataSpringBootAPI)

기능	Method endpoint	요청 형식	응답 형식
회원별 예측 이력 조회	GET /modeldata/view/{mem_id}	Path: mem_id	Model[]
오늘 예측 여부 확인	GET /modeldata/check/{mem_id}	Path: mem_id	Boolean
오늘 예측값 조회	GET /modeldata/today/{mem_id}	Path: mem_id	Double

웹 서비스 개발

API 인터페이스 명세(Flask - FlaskMLAPI)

기능	Method endpoint	요청 형식	응답 형식
심혈관질환 예측	POST /heart_predict	Body: features[]	{ "probability": 0.1234 }

웹 서비스 개발

주요 기능

- 분석 - 심근경색/협심증 위험도 예측

심근경색 발생 확률 구하기

고혈압 의사진단여부 : ☐ 있음 ☐ 없음

이상지질혈증 의사진단여부 : ☐ 있음 ☐ 없음

당뇨병 의사진단여부 : ☐ 있음 ☐ 없음

뇌졸중 의사진단여부 : ☐ 있음 ☐ 없음

고혈압 유형여부 : ☐ 정상 ☐ 주의혈압 ☐ 고혈압전단계 ☐ 고혈압

공복 혈당 : mg/dL

당화혈색소 : %

총콜레스테롤 : mg/dL

체질량지수(BMI) : kg/m²

허리둘레 : cm

평생 일반담배 흡연 여부 : ☐ 5갑(100개비) 미만 ☐ 5갑(100개비) 이상 ☐ 피운적 없음

1년간 음주 빈도 : ☐ 최근 1년간 전혀 마시지 않았다. ☐ 월 1회미만 ☐ 월 1회정도
☐ 월 2~4회정도 ☐ 주 2~3회정도 ☐ 주 4회정도이상

한번에 마시는 음주량 : ☐ 1~2잔 ☐ 3~4잔 ☐ 5~6잔
☐ 7~9잔 ☐ 10잔 이상

하루에 앉아서 보내는 시간 : 시간

유산소 신체 활동 실천율 : ☐ 실천하지않음 ☐ 실천함

성별 : ☐ 남자 ☐ 여자

나이 : 세

최종 학력 : ☐ 초졸 이하 ☐ 중졸 ☐ 고졸 ☐ 대졸이상

소득 분위수 : ☐ 하 ☐ 중하 ☐ 중상 ☐ 상

예측하기

웹 서비스 개발

주요 기능

- 분석 - 심근경색/협심증 위험도 예측

```
const handleSubmit = async(e) => {
  e.preventDefault();
  const loginId = user.mem_id;
  try {
    const checkres = await checkTodayPredict(loginId);
    console.log("체크 응답:", checkres);
    console.log("체크 응답 data:", checkres.data);
    console.log("타입:", typeof checkres.data);

    if(checkres.data === true) {
      alert("오늘은 이미 예측을 완료했습니다.\n예측은 하루에 한 번만 가능합니다.\n결과창으로 이동합니다.");
      const todayRes = await getTodayPredict(loginId);
      navigate("/result", {state: { result: { probability: todayRes.data } }});
      return;
    }

    const requestData = { features: features, mem_id: loginId };
    const res = await getHeartPredict(requestData);
    alert("예측성공 : " + res.data.probability * 100 + "%");
    navigate("/result", {state: { result: { probability: res.data.probability } }});
  } catch(err){
    console.error("예측중 오류 발생 : ", err);
    alert("예측실패");
  }
}
```

예측하기 → handleSubmit 호출

→ requestData를 매개변수로 getHeartPredict(requestData) 호출

→ 백엔드 서버(Flask) /flask/heart_predict 으로 요청



```
export const getHeartPredict = (features) => {
  flaskApi.post("/heart_predict", features)
}
```

사용자 아이디(mem_id) + 사용자에게 입력받은 데이터(features)
= requestData

웹 서비스 개발

주요 기능

- 분석 - 심근경색/협심증 위험도 예측

```
@app.route("/heart_predict", methods=["POST"])
def getheartPred():
    print("=====getheartPredict=====")

    data = request.get_json()
    print(">>>>>전송받은 파라미터 request.get_json() : ", data)

    if isinstance(data, list):
        features = data
        mem_id = "no_enter"
    elif isinstance(data, dict) and "features" in data:
        features = data["features"]
        mem_id = data.get("mem_id", "no_enter")
    else:
        return jsonify({"error": "잘못된 입력 형식입니다."}), 400

    # 19개 데이터를 받아 내부적으로 35개 확장 후 예측 확률 반환
    result = getheartPredict(features)
    print(f"예측 결과 : {result}")

    # DB 저장 (원본 19개와 결과 확률값 1개 저장)
    try:
        save_prediction(mem_id, features, result)
    except Exception as e:
        print(f"저장 오류 : {e}")

    return jsonify({"probability": result})
```

/flask/heart_predict로 요청이 들어오면
 → 사용자가 입력한 데이터를 매개 변수로
 getHeartPredict(features) 호출
 → 예측 결과 반환
 → 사용자가 입력한 19개 컬럼 데이터,
 사용자 아이디, 예측 결과를
 DB에 저장
 : save_prediction(
 mem_id, features, result)



```
def save_prediction(mem_id, features, result):
    conn = get_connection()
    cursor = conn.cursor()
    try:
        sql = """
            INSERT INTO data_test (
                mem_id, check_date,
                di1_dg, di2_dg, de1_dg, di3_dg, he_hp, he_glu, he_hba1c, he_chol,
                he_bmi, he_wc, bs1_1, bd1_11, bd2_1, pa_aerobic, be8_1,
                sex, age, edu, incm,
                predict
            ) VALUES (
                :1, SYSDATE,
                :2, :3, :4, :5, :6, :7, :8, :9,
                :10, :11, :12, :13, :14, :15, :16,
                :17, :18, :19, :20, :21
            )
        """
        params = [mem_id] + features + [result]

        cursor.execute(sql, params)
        conn.commit()
        print("[DB 성공] 오라클 명세서 규격에 맞춰 안전하게 저장 완료되었습니다.")
    except:
```

웹 서비스 개발

주요 기능

- 분석 - 심근경색/협심증 위험도 예측

```
# 1. 모델과 웹 서버용 35개 컬럼 리스트 로드
heart_model = joblib.load("./models/heart_model_last.pkl")
model_features = joblib.load("./models/model_features.pkl") # 아까 저장한 35개 컬럼 리스트

def getheartPredict(features):
    print(">>>>>> 예측 시작 (확률 출력 모드)")

    # 2. 프론트에서 준 19개 변수 순서대로 딕셔너리 매핑
    # (주의: DB에 넣으실 때 선언한 순서와 반드시 일치해야 합니다!)
    feature_names_19 = [
        'di1_dg', 'di2_dg', 'de1_dg', 'di3_dg', 'he_hp', 'he_glu', 'he_hba1c', 'he_chol',
        'he_bmi', 'he_wc', 'bs1_1', 'bd1_11', 'bd2_1', 'pa_aerobic', 'be8_1',
        'sex', 'age', 'edu', 'incm'
    ]

    # 1행짜리 원본 데이터프레임 생성
    df_raw = pd.DataFrame([features], columns=feature_names_19)

    # 3. 범주형 변수들을 위한 인코딩 (pd.get_dummies)
    # 국건영 범주형 데이터 변수들 지정
    cate_cols = ['di1_dg', 'di2_dg', 'de1_dg', 'di3_dg', 'he_hp', 'bs1_1', 'bd1_11', 'bd2_1', 'pa_aerobic', 'sex', 'edu', 'incm']
    categories_dict = {"di1_dg" : [0, 1], "di2_dg" : [0, 1], "de1_dg" : [0, 1], "di3_dg" : [0, 1], "he_hp" : [1, 2, 3, 4], "bs1_1" : [1, 2, 3],
        "bd1_11" : [1, 2, 3, 4, 5, 6], "bd2_1" : [1, 2, 3, 4, 5], "pa_aerobic" : [0, 1], "sex" : [1, 2], "edu" : [1, 2, 3, 4], "incm" : [1, 2, 3, 4]}
    for col in cate_cols:
        if col in categories_dict:
            df_raw[col] = pd.Categorical(df_raw[col], categories=categories_dict[col])

    df_encoded = pd.get_dummies(df_raw, columns=cate_cols, drop_first=True)

    # 4. [핵심] 35개 컬럼 틀에 강제로 맞추기
    # 누락된 범주형 컬럼은 0으로 채우고 순서를 완전히 일치시킵니다.
    df_final = df_encoded.reindex(columns=model_features, fill_value=0)

    # 5. predict_proba()로 확률 계산
    heart_prob = heart_model.predict_proba(df_final)[0]

    # 6. 심근경색(1)일 확률 추출 및 반올림
    heart_positive_prob = round(float(heart_prob[1]), 4)

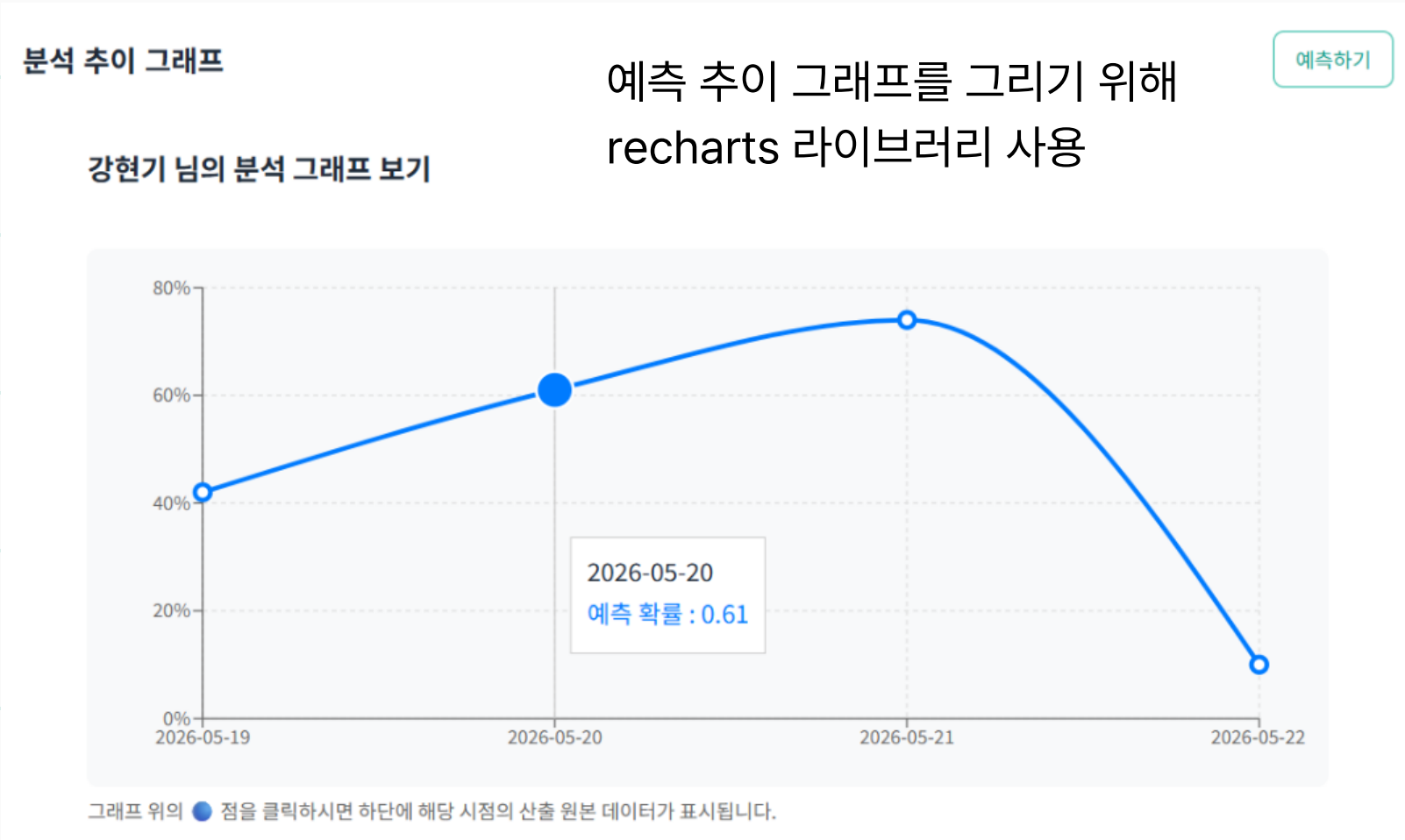
    print(">>>>>> 예측 확률 결과 (1일 확률) : ", heart_positive_prob)
    return heart_positive_prob
```

getHeartPredict 함수 :
사용자가 입력한 19개 컬럼 중
범주형 데이터 컬럼들을 One-Hot 인코딩
(pd.get_dummies)
→ 19개 컬럼에서 35개 컬럼으로 늘어남
→ 학습 시 사용한 35개 컬럼 순서에 맞게 재정렬(reindex())
→ 사용자가 입력한 데이터로 심근경색/협심증 위험도 예측 후 반환

웹 서비스 개발

주요 기능

- 분석 - 예측결과 보기



선택된 시점의 예측 결과 및 산출 데이터 (Raw Data 25개 전체 변수)

검사 일자 (CHECK_DATE): 2026-05-21 머신러닝 예측 확률 (PREDICT): 74%

확률 산출 근거 전체 데이터 목록

데이터 항목 (변수명)	측정 수치 (값)	데이터 항목 (변수명)	측정 수치 (값)
나이 (AGE)	27세	성별 (SEX)	남성
공복혈당 (HE_GLU)	126.5 mg/dL	당화혈색소 (HE_HBA1C)	6.5 %
총콜레스테롤 (HE_CHOL)	220.5 mg/dL	비만도 (HE_BMI)	27.3
허리둘레 (HE_WC)	92.5 cm	고혈압 유병여부 (HE_HP)	4(코드식)
유산소 운동 실천 여부 (PA_AEROBIC)	0(코드식)	주간 평균 수면 시간 (BE8_1)	9 시간
소득 수준 (INCM)	3 분위	교육 수준 (EDU)	4 단계
당뇨병 의사진단 여부 (DI1_DG)	질환/이상	고혈압 의사진단 여부 (DI2_DG)	질환/이상
이상지질혈증 의사진단 (DI3_DG)	정상	뇌졸중 의사진단 여부 (DE1_DG)	질환/이상
1년간 음주 빈도 (BD1_11)	5 (코드식)	한번에 마시는 음주량 (BD2_1)	3 (코드식)
일생동안의 흡연 상태 (BS1_1)	2 (코드식)		

그래프 상의 점 클릭 시 해당 예측 시점의
사용자 입력값을 볼 수 있게 처리

```
const handleClick = (payload) => {
  if (payload?.payload) {
    const data = payload.payload;
    setSelectedData(data);
    const heartResult = calcHeartAgeFromSelectedData(data, NON_SMOKER, SMOKER);
    setHeartAge(heartResult);
  }
};
```


웹 서비스 개발

주요 기능

- 커뮤니티 기능

커뮤니티 인기글

운동 공유

헬스장 추천 부탁드립니다

11

운동 공유

오늘 하체 조졌습시다

10

식단 이야기

벌크업 식단 후기 제보

5

운동 공유

홈트레이닝 추천!!!

3

더보기 >

카테고리

전체 게시물

자유 게시판

건강 정보

식단 이야기

운동 공유

전체 게시물

검색어를 입력하세요

확인

최신순

좋아요순

조회수순

글 작성하기

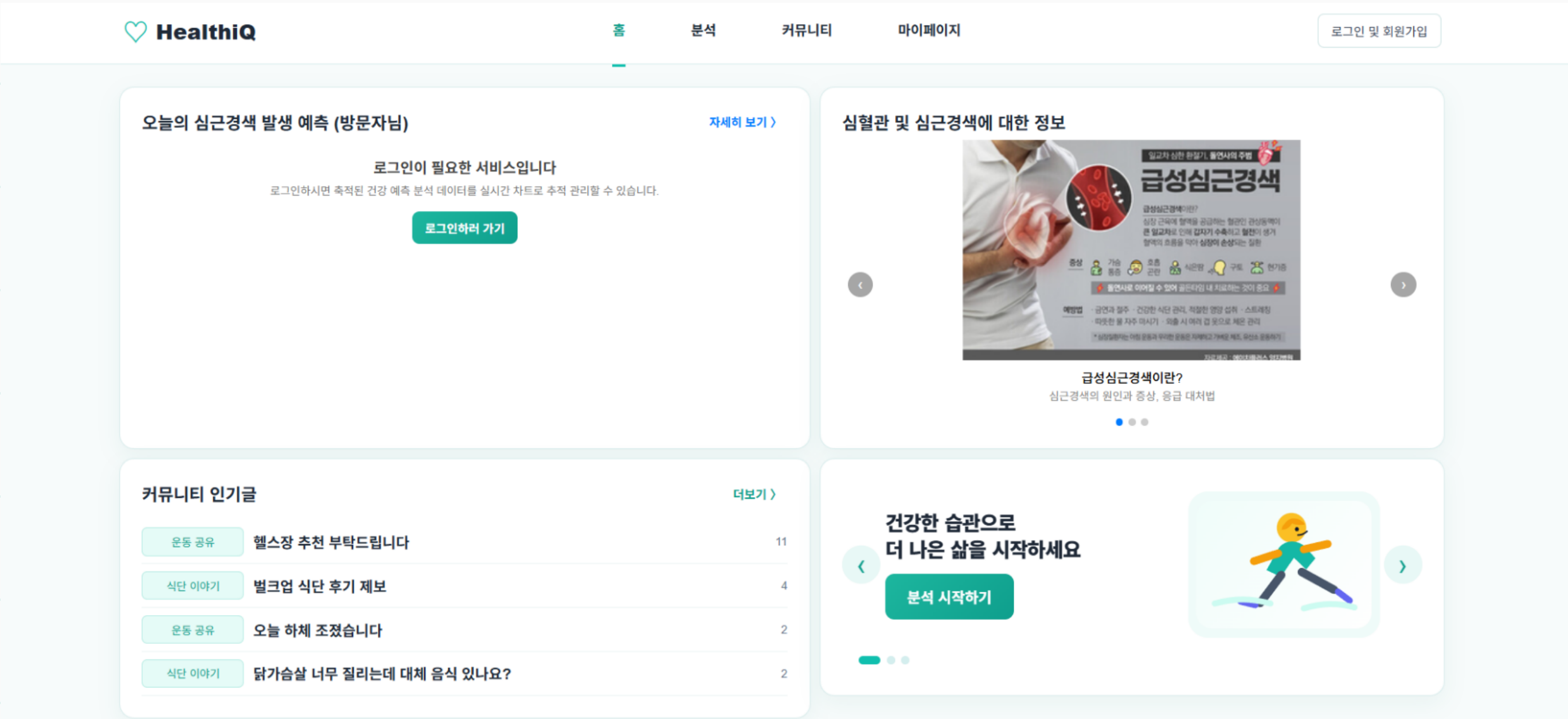
제목	작성자	작성일	조회수	좋아요수
헬스장 추천 부탁드립니다	user1	2026-05-18	155	11
홈트레이닝 추천!!!	shintra0070@gmail.com	2026-05-19	59	2
벌크업 식단 후기 제보	shintra0070@gmail.com	2026-05-22	34	4
운동 루틴 추천	shintra0070@gmail.com	2026-05-20	16	1
오운완 인증	user1	2026-05-19	14	1

< 1 2 3 4 >

현재 1 페이지 / 총 페이지 수 4 페이지

웹 서비스 개발

시연



4. 프로젝트 결론

결론 및 제언

활용 방안

기대 효과

Q&A

Keyword

결론 및 제언

결론

데이터 분석, 모델링 및 예측		웹 서비스 개발 및 기능 구현	
최적 모델 선정	Random Forest, XGBoost, LightGBM 모델을 비교 분석한 결과, XGBoost 가 최적의 알고리즘으로 선정	시스템 아키텍처 및 각 서버 간 연동	React 프론트엔드, Spring Boot 백엔드 서버, Flask 머신러닝 서버 및 Oracle 데이터베이스 간의 연동 구현
최종 예측 모델 성능	GridSearchCV를 통해 도출된 최적의 하이퍼파라미터 조합을 적용하여 최고 ROC-AUC 점수 0.9801, 정확도 0.90을 달성함으로써 위험도 예측 유의성을 확보		
주요 독립변수 유의성	카이제곱 검정 및 Mann-Whitney U test를 수행한 결과, 선정된 19개의 범주형 및 수치형 변수 모두 p-value 0.0000으로 통계적 유의성을 검증 변수 중요도 분석 결과 이상지질혈증 의사진단 여부(DI2_dg), 평생 일반담배 흡연 여부(BS1_1), 연령(age) 등이 예측에 높은 기여도를 나타냄	사용자 기능 제공	사용자가 입력한 데이터를 이용하여 실시간으로 발병 확률을 반환하는 예측 파이프라인 구현 일별 데이터 누적을 통한 Recharts 기반 분석 추이 그래프 시각화 및 게시판 중심의 커뮤니티 환경을 구축

결론 및 제언

한계점

데이터 분석, 모델링 및 예측		웹 서비스 개발 및 기능 구현	
클래스 불균형	소수 클래스에 대한 정밀도(Precision: 0.34)가 재현율(Recall: 0.68) 대비 낮게 산출되는 경향이 존재	사용성 문제	예측 결과를 알기 위해 사용자의 혈액 검사 결과가 필요함 예측 확률만 제공되어 결과가 나온 이유를 사용자가 알기 어려움 모델 성능 지표가 서비스 화면에서 충분히 강조되지 않음
이상치 처리	극단적 이상치에 대한 세분화된 처리 기준 수립의 부재	보안 문제	인증·인가 구조가 실제 서비스 수준으로 완성되지 않음 DB 접속 정보, OAuth Key 등 민감 정보 관리가 미흡

결론 및 제언

제언

데이터 분석, 모델링 및 예측		웹 서비스 개발 및 기능 구현	
클래스 불균형 데이터 보정 지속	수집된 국민건강영양조사 데이터의 클래스 불균형을 해결하기 위해 SMOTE 샘플링을 적용하였으나, 소수 클래스에 대한 정밀도(Precision: 0.34)가 재현율(Recall: 0.68) 대비 낮은 결과를 보임 향후 모델의 오진율(False Positive)을 낮추기 위해 임계값(Threshold) 최적화 및 추가적인 데이터 확보가 요구됨	예측 결과에 대한 신뢰도 향상	주요 변수 영향도와 모델 성능 지표를 시각화해 예측 결과에 대한 신뢰도 확보
수치형 데이터 전처리 고도화	공복혈당 수치 등 일부 이상치에 대해 교차 검증을 거쳐 초고위험군 환자의 실제 데이터로 판단하여 정제 없이 진행하였으나, 향후 모델의 일반화 성능 향상을 위해 극단적 이상치에 대한 세분화된 처리 기준 수립이 필요함	개인 맞춤 피드백	위험 요인별 개선 가이드와 생활습관 관리 메시지를 더욱 세밀하게 제공

프로젝트 결론

활용 방안



지역 사회 연계
보건소·공공 예방 프로그램 활용



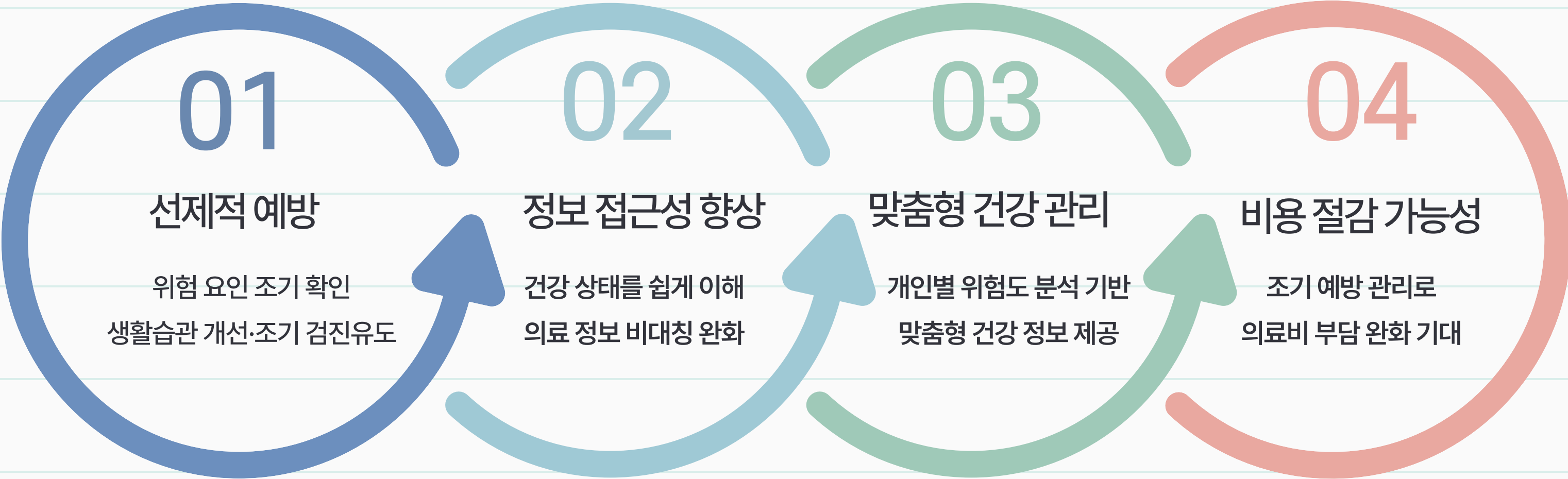
사전 문진 자료
진료 전 건강 상태 참고자료 활용



기업 건강관리
직장인 자가 점검·생활습관 개선

프로젝트 결론

기대 효과



Q&A